

«Talento Tech»

React JS

Clase 07



Clase N° 7 | Rutas estáticas y dinámicas

Índice

Clase N° 7 | Rutas estáticas

Introducción al ruteo en SPAs

Creando nuestra barra de navegación:

Instalación y configuración de react router

Configuración:

Rutas estáticas y estructura principal

<Routes> y <Route>.

De props.children a <Outlet />

El Mecanismo Detrás de Escena

Conectando a nuestra barra de navegación.

Rutas Dinámicas

Ejercicio:

Reflexión Final

Materiales y Recursos Adicionales:

Preguntas para Reflexionar:

Próximos Pasos:

Objetivos de la Clase:

- Instalar y configurar la librería react-router-dom en nuestro proyecto.
- Comprender el funcionamiento del ruteo en una Aplicación de Página Única (SPA).
- Implementar un sistema de navegación principal utilizando rutas estáticas y el componente Link.

Introducción al ruteo en SPAs



React Router

Hasta ahora, nuestra aplicación y todos sus componentes viven en una sola "página" o vista. Si quisiéramos mostrarlas por separado o incluso distintas secciones como "Inicio", "Productos destacados" o "Contacto", la solución tradicional sería crear archivos HTML separados (index.html, productos.html, etc.). Esto, sin embargo, implica que el navegador

debe recargar toda la página cada vez que el usuario navega, perdiendo el estado de nuestra aplicación y resultando en una experiencia de usuario menos fluida.

Las **Single Page Applications (SPAs)** resuelven este problema. En lugar de recargar la página, React intercepta la navegación y simplemente renderiza los componentes necesarios para la nueva "vista" de forma dinámica. El resultado es una experiencia rápida y fluida, similar a la de una aplicación de escritorio o móvil.

Para gestionar esta lógica necesitamos tomar una referencia para cada componente, y esta referencia será la URL. Para eso utilizamos una librería externa muy popular y estándar de facto en el ecosistema de React: **React Router**.

Creando nuestra barra de navegación:

En el desafío de la clase 3 nos pedía incorporar la barra de navegación a nuestro Header. Si todavía no lo hiciste, podés tomar el siguiente código de una barra de navegación muy básica para que puedas practicar los temas de esta clase

En esta oportunidad vamos a necesitar al menos 4 items:

Inicio, Productos, Destacados y Alta de producto, cada uno corresponde a un componente que estuvimos haciendo.

Detalle: Asegurate de que la barra de navegación tenga su estilo correspondiente.

```
import styles from './Header.module.css';

function Header() {
  return (
    <header className={styles.header}>
      <nav>
        <ul>
          <li><a href="">Inicio</a></li>
          <li><a href="">Productos</a></li>
```



```

        <li><a href="">Destacados</a></li>
        <li><a href="">Alta de producto</a></li>
      </ul>
    </nav>
  </header>
);
};

export default Header;

```

Instalación y configuración de react router

El primer paso es añadir la librería a nuestro proyecto:

1. Abrimos la terminal en la raíz de nuestro proyecto.
2. Si nuestra aplicación está corriendo, entonces ejecutamos `ctrl + c` para pararla
3. En la terminal ejecutamos el siguiente comando: **`npm install react-router-dom`**
4. Una vez instalado vamos a encontrar este paquete en la carpeta de `node_modules`.

Configuración:

Para usarlo, debemos "envolver" nuestra aplicación para que toda ella tenga acceso a las funcionalidades de ruteo. Generalmente hacemos esto en el archivo principal, **main.jsx**.

En este archivo vamos a:

1. importar BrowserRouter
`import { BrowserRouter } from 'react-router-dom';`
2. Reemplazamos el `StrictMode` que envolvía nuestra App, por `BrowserRouter`.

```

// main.jsx
import { StrictMode } from 'react' // Podemos eliminarlo.
// Demas importaciones. Importamos BrowserRouter
import { BrowserRouter } from 'react-router-dom';

createRoot(document.getElementById('root')).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>)

```

El componente `BrowserRouter` es el que conecta nuestra aplicación con la API de historial del navegador, permitiéndonos mantener la UI sincronizada con la URL.

Rutas estáticas y estructura principal

Ahora que hemos sentado las bases, es momento de definir la arquitectura de navegación de nuestra aplicación. Lo haremos dentro de nuestro componente principal, `App.jsx`. Para ello, nos valdremos de dos componentes fundamentales que nos provee React Router:



`<Routes>` y `<Route>`.

- `<Routes>` (plural):** Este componente actúa como un contenedor principal. Su función es examinar todas las definiciones de rutas hijas (`<Route>`) que contiene y renderizar únicamente la primera página cuya propiedad `path` coincida con la URL actual del navegador. Es, en esencia, el orquestador del ruteo.
- `<Route>` (singular):** Representa una ruta individual en nuestra aplicación. Es el componente que establece la correspondencia directa entre un segmento de la URL y el componente de React que debe visualizarse. Sus propiedades (props) más importantes son:
 - `path`:** Una cadena de texto (string) que define el patrón de la URL. Por ejemplo, `"/productos"`.
 - `element`:** Recibe el componente (en formato JSX, por ejemplo `<Productos />`) que se renderizará cuando la URL coincida con el `path` definido.

Para estructurar nuestra aplicación, reutilizaremos los componentes que hemos desarrollado en clases anteriores, asignando a cada uno una "página" o vista específica:

- `ItemListContainer`:** Será nuestra vista de productos principal, donde inicialmente mostrábamos datos directamente desde el código.
- `Productos`:** Funcionará como una vista de productos "destacados", que consume datos desde un archivo local, simulando una llamada a una API.
- `FormularioContainer`:** Corresponderá a la sección de "alta de productos", donde gestionamos la lógica para capturar datos del usuario.

Rutas anidadas en App.jsx

Nuestro objetivo es lograr una estructura donde el Header y el Footer, contenidos en nuestro Layout, permanezcan visibles en todas las páginas, mientras que el contenido principal sea dinámico y cambie según la navegación del usuario.

Para eso, comenzaremos importando los componentes necesarios en App.jsx:

```
// /src/App.jsx
import './App.css';
import { ItemListContainer } from
  './componentes/ItemListContainer/ItemListContainer';
import { Contador } from './componentes/Contador/Contador' // Lo podemos
eliminar.
import Productos from './componentes/Productos/Productos';
import { FormularioContainer } from
  './componentes/FormularioContainer/FormularioContainer'
import { Routes, Route } from 'react-router-dom';
import Layout from './componentes/layout/Layout';
```

Definiremos una ruta principal que renderizará nuestro <Layout /> y, dentro de ella, anidaremos las rutas específicas para cada una de nuestras "páginas".

```
// todas las importaciones anteriores
function App() {
  return (
    <Routes>{ /*envuelve a las demás para mostrar Header y Footer siempre */}
      <Route element={<Layout />}>
        <Route path="/" element={<h1>Página de Inicio</h1>} />
        <Route path="/productos" element={<ItemListContainer
          Mensaje={"Productos destacados"} />} />
        <Route path="/destacados" element={<Productos
          Mensaje={"Todos los productos"} />} />
        <Route path="/alta" element={<FormularioContainer />} />
      </Route>
    </Routes>);
}
```

Nota importante: Observen que estamos anidando las rutas dentro de una ruta contenedora que renderiza nuestro Layout. Todavía no funciona! porque para que esto funcione, el componente Layout debe incluir el componente **Outlet** de React Router, que actúa como un marcador de posición donde se renderizarán los componentes hijos (Productos, AltaProductos, etc.).

Integración de nuestro layout con Outlet

Pensemos en nuestro componente `<Layout />` como el **marco de un cuadro**. Este marco es constante y define la estructura visual que siempre estará presente: nuestro encabezado (Header) y nuestro pie de página (Footer).

Ahora, el contenido de ese cuadro —la pintura— es lo que necesita cambiar dinámicamente según la sección de nuestra aplicación que el usuario esté visitando. React Router usa `<Outlet />` donde debe colocar esa "pintura" dentro de nuestro marco

De `props.children` a `<Outlet />`

En un componente de React tradicional, sin ruteo, usaríamos la prop `children` para pasar contenido. Haríamos algo como esto:

```
// En App.jsx (SIN React Router)
<Layout>
  <h1>Este es el contenido de la página de inicio</h1>
</Layout>

// En Layout.jsx
function Layout({ children }) {
  return (
    <>
      <Header />
      <main>{children}</main> {/* Aquí renderizamos lo que pasamos */}
      <Footer />
    </>
  );
}
```

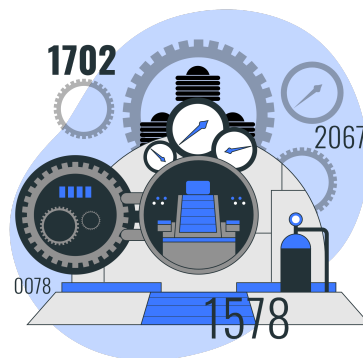
Este patrón funciona, pero es estático. Sin embargo, con React Router, la situación es diferente porque el componente que se debe renderizar no lo decide App directamente, sino que lo determina la URL actual del navegador.

El componente `<Outlet />` actúa como un marcador de posición dinámico y contextual. No es una prop, sino un componente provisto por React Router que está sabe del estado del ruteo.

El Mecanismo Detrás de Escena

Cuando un usuario navega a, por ejemplo, /productos:

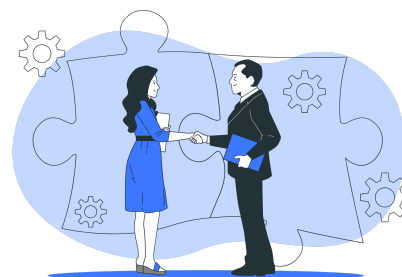
1. **Coincidencia de Ruta:** El componente `<Routes>` en `App.jsx` analiza la URL. Encuentra que `/productos` coincide con una ruta anidada: `<Route path="/productos" element={}<ItemListContainer /> />`.
2. **Renderizado del Padre:** React Router observa que esta ruta está dentro de la ruta contenedora `<Route element={}<Layout /> />`. Por lo tanto, lo primero que hace es renderizar el elemento de la ruta padre: el componente `<Layout />`.
3. **El Rol del Outlet:** Mientras renderiza `<Layout />`, se encuentra con el componente `<Outlet />`. En ese preciso momento, `<Outlet />` consulta el estado interno de React Router y dice: "Ah, estoy dentro de un layout que corresponde a una ruta padre. La ruta hija que coincidió es `/productos`, por lo tanto, en mi lugar, debo renderizar el elemento correspondiente a esa ruta, que es `<ItemListContainer />`".



```
// En /componentes/layout/Layout.jsx
import Header from '../layout/Header'
import Footer from '../layout/Footer'
import { Outlet } from 'react-router-dom'; // Importamos Outlet
// Todo lo que pongamos dentro de <Layout> en App.jsx será el "children".
function Layout({ children }) {
  return (
    <>
      <Header />
      <main>
        {/* Aquí se renderizará el componente de la ruta activa */}
        <Outlet />
      </main>
      <Footer />
    </>
  );
};
export default Layout;
```


Conectando a nuestra barra de navegación.

Ya tenemos las rutas definidas y necesitamos que los enlaces de nuestra barra de navegación (el componente Header.jsx de la clase 3) realmente naveguen a esas rutas sin recargar la página. Para esto, reemplazamos las etiquetas `<a>` por el componente **Link** de React Router.



Header.jsx - Reemplazando `<a>` por `<Link>`

La diferencia es fundamental: estará en que la etiqueta `<a href="/productos">` le dice al navegador "andá a buscar este documento", causando una recarga completa. En cambio, `<Link to="/productos">` le dice a React Router "cambiá la URL y renderizá el componente asociado a esta ruta", sin recargar la página.

```
import styles from './Header.module.css';
import { Link } from 'react-router-dom'; // 1. Importamos Link
function Header() {
  return (
    <header className={styles.header}>
      <nav>
        <ul>
          { /* 2. Usamos Link en lugar de <a> y 'to' en lugar de 'href' */ }
          <li><Link to="/">Inicio</Link></li>
          <li><Link to="/productos">Productos</Link></li>
          <li><Link to="/destacados">Destacados</Link></li>
          <li><Link to="/alta">Alta de Productos</Link></li>
        </ul>
      </nav>
    </header>
  );
};
export default Header;
```

Rutas Dinámicas

Vimos que nuestra página ya tiene el ruteo correcto, pero tenemos acceso a todos los componentes que enrutemos. Sin embargo muchas veces vamos a necesitar ver una información y no otra, como también tener acceso a información puntual de un producto.

Rutas Dinámicas

¿Qué sucede si queremos tener una página de detalle para cada producto? No vamos a crear una ruta estática por cada uno. En su lugar, creamos una ruta dinámica que utilice un parámetro.

En **App.jsx**, agregamos una nueva ruta con un parámetro `:id`:

```
import ProductoDetalle from '../componentes/Productos/ProductoDetalle';
{/* ... dentro de <Routes> */}
  <Route path="/producto/:id" element={<ProductoDetalle />} />
```

El prefijo `:` (dos puntos) le indica a React Router que `id` es un parámetro dinámico. Para acceder al valor de este parámetro dentro del componente `ProductoDetalle`, usamos el hook **useParams** que nos devuelve un objeto con los parámetros de la URL.

ProductoDetalle.jsx - Accediendo a Parámetros de URL

```
import { useParams } from 'react-router-dom';

const ProductoDetalle = () => {
  const { id } = useParams();

  // Con este 'id', podríamos hacer una llamada a una API para buscar los
  // datos del producto

  return (
    <div>
      <h2>Detalle del Producto</h2>
      <p>Mostrando información para el producto con ID:
      <strong>{id}</strong></p>
    </div>
  );
};

export default ProductoDetalle;
```

Resultado

Vamos a probar nuestra función. Por el momento vamos generar la URL manualmente, ingresando por ejemplo:

<http://localhost:5173/producto/6>

Si ingresamos esperamos ver un párrafo con la siguiente información:

Mostrando información para el producto con ID: 6

Mejora

Vamos a mejorar nuestra aplicación y hagamos que al cambiar nuestra URL, nos muestre toda la información del producto en cuestión.

```
import { useState, useEffect } from 'react';
import { useParams } from 'react-router-dom';

const ProductoDetalle = () => {
  const { id } = useParams();
  const [producto, setProducto] = useState(null);

  useEffect(() => {
    fetch('/data/productos.json')
      .then(response => response.json())
      .then(data => {
        const productoEncontrado = data.find(p => p.id === parseInt(id));
        setProducto(productoEncontrado);
      })
      .catch(error => console.error("Error al cargar el producto:", error));
  }, [id]);

  if (!producto) {
    return <h2>Cargando detalle del producto...</h2>;
  }

  if (!producto.id) {
    return <h2>Producto no encontrado.</h2>;
  }

  return (
    <div>
      <h2>Detalle del Producto: {producto.nombre}</h2>
    </div>
  );
}
```

```

      <img src={producto.imagen} alt={producto.nombre} style={{ maxWidth:
'400px' }} />
      <h3>${producto.precio}</h3>
      <p>{producto.descripcion}</p>
    </div>
  );
};

export default ProductoDetalle;

```

linkeando productos

Vamos ahora a editar nuestro componente Producto.jsx para que podamos darle clic a un producto y se modifique nuestra URL.

En estos casos, vamos igualmente a incorporar la etiqueta Link, pero además de importarla vamos a utilizar otra forma para pasarle el id.

<Link to={` /producto/\${producto.id}`}></Link>

```

import React, { useState, useEffect } from 'react';
import styles from './Productos.module.css';
import { Link } from 'react-router-dom';
function Productos({Mensaje}) { // Los useState se mantienen igual
  useEffect(() => { // Con su fetch que llama a data/json }
    return (
      <div>
        <h1>{Mensaje}</h1>
        <ul className={styles.listaProductos}>
          {productos.map((producto) => (
            <li key={producto.id} className={styles.itemProducto}>
              // Acá está lo importante. Prestá atención a la etiqueta <Link></Link>
              <Link to={` /producto/${producto.id}`}>
                <h2>{producto.nombre}</h2>
                <img src={producto.imagen} alt={producto.nombre} width="150" height="150" />
                <p>Precio: ${producto.precio}</p>
              </Link></li>))}
            </ul>
          </div>
        );
      )
    };
  }
export default Productos;

```

Reflexión Final

En esta clase hemos incorporado una de las herramientas más fundamentales del desarrollo con React: React Router. Aprendimos a estructurar nuestra aplicación en múltiples vistas, a crear una navegación fluida y sin recargas con el componente Link. Dominar el ruteo nos permite pasar de crear componentes aislados a construir aplicaciones web completas, robustas y con una experiencia de usuario profesional. Esta habilidad es la base para desarrollar prácticamente cualquier aplicación de React de una complejidad media o alta.

TalentoLab - Proyecto final



Descripción de tu tarea:



¡Venís haciendo un excelente trabajo! Para aplicar los conceptos de esta clase, tu próxima tarea será implementar la página de inicio de tu proyecto utilizando rutas estáticas para lograr una navegación más fluida y profesional.

Ejercicio:

Construcción de la página de inicio

¡Venís haciendo un excelente trabajo! Para aplicar los conceptos de esta clase, tu próxima tarea será implementar la página de inicio de tu proyecto utilizando rutas estáticas para lograr una navegación más fluida y profesional.

Notarás que en nuestra barra de navegación ya existe un enlace "Inicio" que apunta a la ruta raíz (/). Tu misión es darle vida a esa sección.

Para eso, deberás completar los siguientes puntos:

1. **Crear el componente Inicio.jsx:** Este será el nuevo componente para la página principal. ¡Dale un buen diseño!
2. **Incorporar una sección de "Productos destacados":** Dentro de tu nuevo componente Inicio, deberás mostrar los productos destacados que ya obtenías en clases anteriores.
3. **Incorporar el detalle de cada producto:** Al darle clic a un producto, debemos ver toda la información puntual de ese producto.

Material y Recursos Adicionales:

- ★ [Documentación oficial de React Router](#)
- ★ [React Router - Guía para Principiantes](#)
- ★ [Guía de Rutas Anidadas \(Nested Routes\)](#)

Preguntas para Reflexionar:

1. ¿Cuál es el impacto directo en la experiencia del usuario al implementar un ruteo del lado del cliente (SPA) en lugar de uno tradicional que recarga la página?
2. Al usar rutas dinámicas como `/producto/:id`, ¿qué nuevo desafío introduce el obtener el id desde la URL para el manejo de los datos y el estado del componente?
3. ¿De qué manera el uso de `<Outlet />` junto a rutas anidadas fomenta una arquitectura de componentes más limpia y reutilizable a medida que la aplicación crece?

Próximos Pasos:

- Comprender las limitaciones del manejo de estado local en aplicaciones complejas.
- Aprender a identificar y solucionar el *prop drilling*.
- Implementar la Context API para crear un estado global y accesible desde cualquier componente.
- Centralizar la lógica de nuestro carrito de compras para que sea más mantenible y escalable.
- Poder subir una aplicación React en Vencel y Netlify.



Buenos Aires
aprende
Agencia de Políticas para el Futuro

BA Buenos
Aires
Ciudad